ROS2 Spring Workshop

Lab 6: The Weed Sprayer (Computer Vision & OpenCV)

Project Context: The "Weed Sprayer"

In **Phase 2** of your Final Project, your robot must identify red weed markers on the field, drive over them, and stop to spray.
The robot won't use the God Node to find the markers; it must use its camera to hunt for targets.
This lab builds that vision brain.
Our markers will be **red balls**. The choice of a spherical target is not accidental — more on this in Part 4.

Objective

In this lab, we give the robot eyes. To make development faster, we will run the camera and the code **directly on your laptop**.

You will learn:

- **OpenCV:** The common library for computer vision.
- **HSV Colour Space:** How to detect "red" regardless of lighting.
- **Shape Analysis:** Using math to distinguish a red circle from a red shoe.

Pre-Lab Concept Primer: The Robot Eye

1. What is an Image? (The Spreadsheet)

To us, a picture is a visual scene. To a computer, a picture is just a **spreadsheet**.

- A standard HD image (1920×1080) is just a grid of 2,073,600 pixels.
- Each pixel is a container holding 3 numbers: Blue, Green, Red (0–255).
- **Computer vision** is simply performing math on this giant spreadsheet.

2. The "Colour Space" Trap (RGB vs. HSV)

This is the most common mistake in robotics.

- **RGB (Red-Green-Blue):** This is how *screens* work. It mixes light. If a shadow falls on a red apple, the RGB numbers change completely (e.g., $(255, 0, 0) \rightarrow (100, 0, 0)$). The robot goes blind.
- **HSV (Hue-Saturation-Value):** This is how *humans* describe colour. By looking only at the **Hue (H)** value, we can tell the robot "look for Hue ≈ 0 (red)" regardless of how bright or dark the room is.

3. The Red Hue Wraparound (Gotcha!)

Red is weird in HSV. The hue wheel is circular, and red sits at BOTH ends: near $H = 0$ and near $H = 180$. If your calibration shows some pixels near $H = 5$ and some near $H = 175$, those are both red — just on opposite sides of the wheel.
The fix: for red specifically, you sometimes need TWO masks (one for each range) and OR them together. For most well-lit red objects, a single range like $[0, 10]$ will work, but if detection is flaky, the wraparound is usually the culprit.

4. Shape Analysis (Circularity)

We use math to distinguish specific shapes from random noise. The **circularity** formula compares area to perimeter:

$$Circularity = \frac{4\pi \cdot Area}{Perimeter^2}$$

The math: A circle has the most efficient area-to-perimeter ratio in geometry — that is what the formula measures.
Worked example: A 30×30 pixel square has area $= 900$ and perimeter $= 120$. Plugging in:

$$Circularity = \frac{4\pi \cdot 900}{120^2} = \frac{11{,}310}{14{,}400} \approx 0.785$$

A circle drawn at the same resolution gives ≈ 0.95 due to pixelation artifacts on the jagged edges.
Reference values:

- Perfect circle: 1.0 (unachievable in pixels)
- Pixelated circle: ~ 0.90 to 0.98
- Square: ≈ 0.785
- Weird blob (shoe/leaf): < 0.5

Part 1 — Laptop Hardware Setup

We will install the drivers directly on your laptop (or VM). These tools allow ROS to communicate with your USB hardware and visualize the data.

1. **Install the camera driver & visualizer:** *Note: We install the camera driver and the RQT image viewer together to save time.*

```
sudo apt-get update
sudo apt-get install ros-humble-usb-cam ros-humble-rqt-image-view

```

2. **Install OpenCV (Python):** *Note: We use the system package manager to ensure compatibility with the ROS 2 environment.*

```
sudo apt-get install python3-opencv

```

3. **The "Hardware Handshake" (Windows Users):** If you are using Docker on Windows, you must manually "pass" the USB device from Windows to the Linux kernel.

   - Open PowerShell (Admin) and run: `usbipd list`
   - Bind and attach your camera's Bus ID (e.g., 1-13):

```
usbipd bind --busid 1-13
usbipd attach --wsl --busid 1-13

```

Heads-Up: Bandwidth & MJPEG

Virtual USB connections (Docker on Windows, especially) have limited bandwidth. Raw video formats like YUYV are huge and cause `Select Timeout` errors. To prevent this, we launch the camera driver with `pixel_format:=mjpeg2rgb` in Part 6. If you see timeout errors there, this is why.

Part 2 — Package Creation

1. Navigate to your source folder:

```
cd /workspaces/agtech_ros2/src

```

2. Create the package:

```
ros2 pkg create --build-type ament_python lab6_vision --dependencies rclpy
    sensor_msgs cv_bridge

```

3. **Import resources:** Copy the starter files from the resources folder into your new package:

```
# Navigate to the Python folder inside your new package
cd /workspaces/agtech_ros2/src/lab6_vision/lab6_vision

# Copy the starter files
cp /workspaces/agtech_ros2/lab_resources/lab6/*.py .

```

Part 3 — The Calibrator Tool

The biggest frustration in computer vision is guessing the numbers. *"Is this red 170? Or 175?"*

Task: Run the `colour_calibrator.py` script (using Python 3 directly).

```
python3 colour_calibrator.py
```

Why Python Directly

The calibrator uses OpenCV's direct webcam access (`cv2.VideoCapture(0)`) instead of subscribing to a ROS topic. This is faster for calibration — no ROS nodes to launch, no topics to remap. Your weed detector (Part 4) will use ROS since it has to integrate with the rest of the robot.

Hold your red ball in front of the camera. Adjust the sliders until the ball is **white** in the mask and the background is **black**.

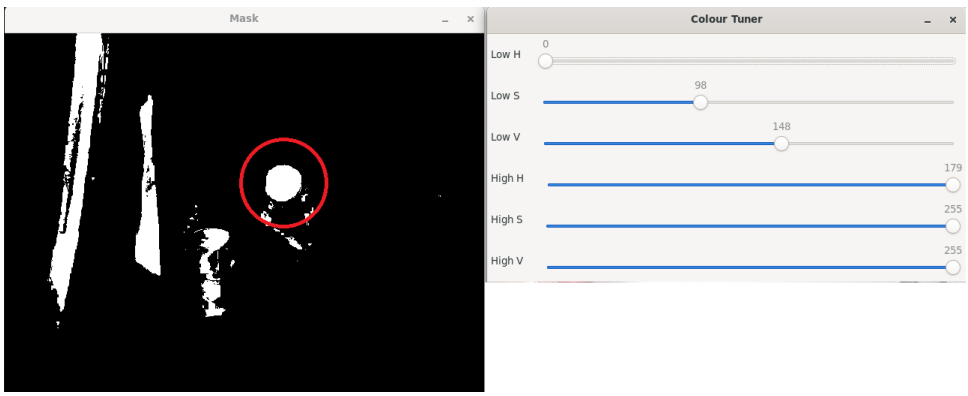

Figure 1: The Colour Calibrator: Tuning the sliders to isolate the marker (white) from the background (black).

Record your values here:

- Low H: _____ S: _____ V: _____
- High H: _____ S: _____ V: _____

Part 4 — The "Weed Detector" Node

This node listens to the camera, filters for red, and uses math to find only circular blobs.

Geometry Insight: Why Red Balls?

A ball (sphere) has a property mathematicians call **point symmetry**: every axis through the centre is a symmetry axis. One practical consequence is that **the silhouette of a sphere projected onto a 2D image is always a circle**, no matter what angle the camera is looking from.
A sphere viewed from above? Circle. From 45 degrees? Circle. From the side? Circle (just smaller).
Compare that to a flat disc: from directly above it's a circle, but from any other angle it becomes an ellipse with circularity dropping as low as 0.5. A camera mounted on a robot looks at the floor from an angle, so flat discs would be a perspective nightmare.
By choosing red balls as our markers, we sidestep the perspective problem entirely. Your code can use a high circularity threshold (> 0.8) and trust it.

Task: Open `weed_detector.py` in VS Code and fill in the blanks.

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import Bool
from cv_bridge import CvBridge
import cv2
import numpy as np
import math

class WeedDetector(Node):
    def __init__(self):
        super().__init__('weed_detector')

        # 1. Subscribe to the camera feed.
        #    The usb_cam driver publishes to /image_raw by default.
        self.sub = self.create_subscription(
            Image, '<TODO_TOPIC>', self.image_callback, 10)

        # 2. Publishers (Debug View & Spray Trigger)
        self.pub_debug = self.create_publisher(
            Image, '/weed_detector/debug', 10)
        self.pub_spray = self.create_publisher(
            Bool, '/tractor/spray', 10)

        self.bridge = CvBridge()

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        except Exception as e:
            return

        # 3. CONVERT TO HSV
        # Note: OpenCV is BGR-native (not RGB!), so we use COLOR_BGR2HSV.
        # Many online tutorials show COLOR_RGB2HSV -- those are for libraries
        # like Pillow or matplotlib. Using the wrong one swaps your red and
        # blue channels and your detector silently looks for blue instead.
        hsv_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)

        # 4. INSERT YOUR CALIBRATED NUMBERS HERE
        lower_red = np.array([ <H>,  <S>,  <V>])
```

```
        upper_red = np.array([ <H>,  <S>,  <V>])

        mask = cv2.inRange(hsv_image, lower_red, upper_red)

        # 5. FIND CONTOURS (Blobs)
        contours, _ = cv2.findContours(
            mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

        found_weed = False

        for contour in contours:
            area = cv2.contourArea(contour)
            perimeter = cv2.arcLength(contour, True)

            # Filter 1: Size (Ignore tiny specks < 500 pixels)
            # 500 px ~= a 22x22 blob. Tune UP if you're far from the marker
            # (the ball will look small in the frame); tune DOWN if you're
            # close. Without this filter, JPEG compression artifacts and
            # small reflections trigger false detections every frame.
            if area > 500 and perimeter > 0:

                # Filter 2: Shape (Circularity)
                # Formula: 4 * PI * Area / Perimeter^2
                circularity = (4 * math.pi * area) / (perimeter * perimeter)

                # Since our markers are red BALLS (spheres), they project
                # to a circle at every viewing angle. We can use a strict
                # threshold and trust it.
                if circularity > 0.8:
                    found_weed = True
                    # Draw Green Outline around the weed
                    cv2.drawContours(cv_image, [contour], -1, (0, 255, 0), 3)

        # 6. PUBLISH TRIGGER
        spray_msg = Bool()
        spray_msg.data = found_weed
        self.pub_spray.publish(spray_msg)

        # 7. PUBLISH DEBUG IMAGE
        self.pub_debug.publish(self.bridge.cv2_to_imgmsg(cv_image, "bgr8"))

def main(args=None):
    rclpy.init(args=args)
    node = WeedDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # SAFETY: explicitly publish spray=False on shutdown so a latched
        # "spraying" signal doesn't outlive this node. In the Final
        # Project this prevents the solenoid from being stuck open
        # after you Ctrl+C your vision code.
        stop_msg = Bool()
        stop_msg.data = False
        node.pub_spray.publish(stop_msg)
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Listing 1: weed_detector.py (Fill in the TODOs)

Code Walkthrough: Chunk by Chunk

Let's break down exactly what this script is doing:

- **Lines 1–9 (Imports):** We start with the usual ROS 2 imports (`rclpy`, `Node`), but three new ones appear. `sensor_msgs.msg.Image` is the standard ROS message type for camera frames — it wraps a raw pixel array with metadata like resolution and encoding. `cv_bridge.CvBridge` is the

translator between ROS's `Image` messages and OpenCV's NumPy arrays; without it, you'd have to manually unpack bytes. Finally, `std_msgs.msg.Bool` is the simplest possible message — a single `True`/`False` — which is all the spray controller needs to know.

- **Lines 11–28 (The Constructor — Wiring the Eyes):** The `__init__` method sets up three connections. First, `create_subscription(Image, ...)` subscribes to the camera feed, just like Lab 3 subscribed to `/turtle1/pose` — but instead of a pose arriving 62 times per second, a full image arrives 30 times per second. Second, we create *two* publishers: one for the debug image (so we can see what the detector sees in `rqt_image_view`) and one for the spray trigger. Third, `self.bridge = CvBridge()` creates the translator we'll use every frame. Notice there is no timer here — the camera *is* the clock. Every time a frame arrives, the callback fires.

- **Lines 30–35 (Frame Conversion):** The callback receives a ROS `Image` message, which is just a flat array of bytes with a header. Line 33 uses `CvBridge` to convert it into a NumPy array in BGR format (OpenCV's default channel order). The `try/except` silently drops corrupted or mismatched frames — on USB cameras this happens occasionally and crashing would kill the whole node.

- **Lines 37–42 (The HSV Transform):** Line 38 converts the BGR image to HSV colour space. This is the key insight from the Concept Primer: in BGR, a shadow changes all three numbers; in HSV, a shadow mostly changes V (brightness) while H (hue) stays nearly constant. Lines 40–41 define the "acceptable red" range using the six numbers you found with the calibrator. Line 42 calls `inRange()`, which returns a binary mask — every pixel that falls inside your HSV range becomes white (255), everything else becomes black (0).

- **Lines 44–46 (Contour Detection):** `findContours()` traces the boundaries of every white blob in the mask. Think of it as drawing an outline around each island of white pixels. It returns a list of contours, where each contour is an array of (x, y) points tracing the blob's edge. The `RETR_TREE` and `CHAIN_APPROX_SIMPLE` arguments control how nested contours are organized and how many points are stored; for our purposes, they just work.

- **Lines 48–60 (The Two-Filter Pipeline):** This is where noise becomes signal. For each contour, we compute its area and perimeter. **Filter 1 (size):** anything under 500 pixels is too small to be our ball — it's a speck of red on a shoe, a reflection, or sensor noise. **Filter 2 (shape):** the circularity formula from the Concept Primer. Because our markers are spheres, their silhouette is always a circle regardless of viewing angle, so we can set a strict threshold of 0.8. A red shoe or leaf would score well below this. Only blobs that pass *both* filters get a green contour drawn around them.

- **Lines 62–68 (Publishing the Decision):** We wrap the boolean `found_weed` in a `Bool` message and publish it to `/tractor/spray`. Any downstream node (your Phase 2 controller, the physical solenoid driver) can subscribe to this topic and act on it. We also publish the annotated image so you can visually confirm detections in `rqt_image_view`. Notice that the spray signal is published *every frame*, not just when a weed is found — this means the topic always reflects the current state of the camera, rather than getting stuck on a stale `True`.

- **Lines 70–83 (The Safe Shutdown):** The `main()` function uses a `try/except/finally` pattern. Under normal operation, `rclpy.spin()` keeps the node alive and processing callbacks indefinitely. When you hit Ctrl+C, the `KeyboardInterrupt` is caught, and the `finally` block runs regardless. It publishes one last `spray=False` before destroying the node. This is a professional safety habit: if your last published message was `True` and you kill the node, that `True` could linger on the wire. On the physical robot, that means the spray solenoid stays open, wasting chemicals or flooding the field.

Professional Habit: Always Zero Outputs On Shutdown

Notice the `try/except/finally` block in `main()`. When you Ctrl+C, the `finally` block publishes `spray=False` before the node exits.
Why it matters: `/tractor/spray` is a **latched** concept — if your last published value was `True`, that "true" sits on the wire. On the physical robot in the Final Project, that could leave the spray solenoid open after you killed your code. Always leave your outputs in a safe state when your code exits.
You used the same pattern in Labs 7 and 8, and you will use it again in Lab 10.

Real-World CV: False Positives

Your detector fires on *anything* red and circular. In the Triathlon arena this is fine because the instructor controls what's on the floor. But in a real farm you would get false positives from:

- Discarded red drink cans (red AND roughly circular from above)
- Wet red leaves flattened by rain
- Ladybugs at close range
- Red tape or paint on tools left in the field

How would you reduce false positives? You do not need to implement this — just think about it. Ideas:

- **Size bounds:** Reject contours that are too large (a soda can) or too small (a ladybug).
- **Temporal filtering:** Require the detection to persist for N frames before firing, to reject single-frame flickers.
- **Aspect ratio check:** The bounding box of a real sphere is close to square (1:1). Reject contours with wildly elongated bounding boxes.
- **Context:** Use the robot's GPS — only trust detections that happen inside known crop rows.

Real agricultural CV engineers spend more time on false-positive rejection than on the initial detection.

Part 5 — Registration & Build

1. Open `setup.py` and add the entry point:

```
'console_scripts': [
    'weed_detector = lab6_vision.weed_detector:main',
],

```

2. Build and source:

```
cd /workspaces/agtech_ros2
colcon build --symlink-install
source install/setup.bash

```

Part 6 — Running the Full System

Robotics requires multiple processes running at once. Open **three separate terminals**.

1. **Terminal 1: The Camera Driver.** This turns on the hardware. We use `mjpeg2rgb` to prevent USB timeouts (the bandwidth issue from Part 1).

```
ros2 run usb_cam usb_cam_node_exe --ros-args -p pixel_format:="mjpeg2rgb"

```

2. **Terminal 2: Your Vision Node.** This runs your logic.

```
source install/setup.bash
ros2 run lab6_vision weed_detector

```

3. **Terminal 3: Visualization.** Open the viewer and select `/weed_detector/debug` from the dropdown.

```
ros2 run rqt_image_view rqt_image_view

```

Deliverables

Submit a single PDF containing:

1. **Calibration:** Screenshot of `colour_calibrator.py` isolating red.
2. **Detection:** Screenshot of `rqt_image_view` showing the green contour around a red ball.
3. **Logic:** Screenshot of terminal echoing `/tractor/spray: True` when a ball is in view.
4. **Reflection (3–5 sentences):** From the False Positives box in Part 4, pick one rejection strategy (size bounds, temporal filtering, aspect ratio, or GPS context) and explain how you would implement it. Which other fruits/vegetables would be problematic for this detector in a real field?

Appendix A: Colour Calibrator Code

Use this code if you cannot access the lab resources.

```
#!/usr/bin/env python3
import cv2
import numpy as np

def nothing(x):
    pass

# Initialize Webcam (0 = Default)
cap = cv2.VideoCapture(0)

cv2.namedWindow('Colour Tuner')

# Sliders for HSV
cv2.createTrackbar('Low H', 'Colour Tuner', 0, 179, nothing)
cv2.createTrackbar('Low S', 'Colour Tuner', 100, 255, nothing)
cv2.createTrackbar('Low V', 'Colour Tuner', 100, 255, nothing)
cv2.createTrackbar('High H', 'Colour Tuner', 10, 179, nothing)
cv2.createTrackbar('High S', 'Colour Tuner', 255, 255, nothing)
cv2.createTrackbar('High V', 'Colour Tuner', 255, 255, nothing)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    l_h = cv2.getTrackbarPos('Low H', 'Colour Tuner')
    l_s = cv2.getTrackbarPos('Low S', 'Colour Tuner')
    l_v = cv2.getTrackbarPos('Low V', 'Colour Tuner')
    u_h = cv2.getTrackbarPos('High H', 'Colour Tuner')
    u_s = cv2.getTrackbarPos('High S', 'Colour Tuner')
    u_v = cv2.getTrackbarPos('High V', 'Colour Tuner')

    lower_bound = np.array([l_h, l_s, l_v])
    upper_bound = np.array([u_h, u_s, u_v])

    mask = cv2.inRange(hsv, lower_bound, upper_bound)

    # Note: the trackbar window 'Colour Tuner' shows the raw frame above
    # the sliders; the 'Mask' window shows what your current thresholds
    # actually isolate. Tune until Mask shows your target as solid white.
    cv2.imshow('Colour Tuner', frame)
    cv2.imshow('Mask', mask)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```